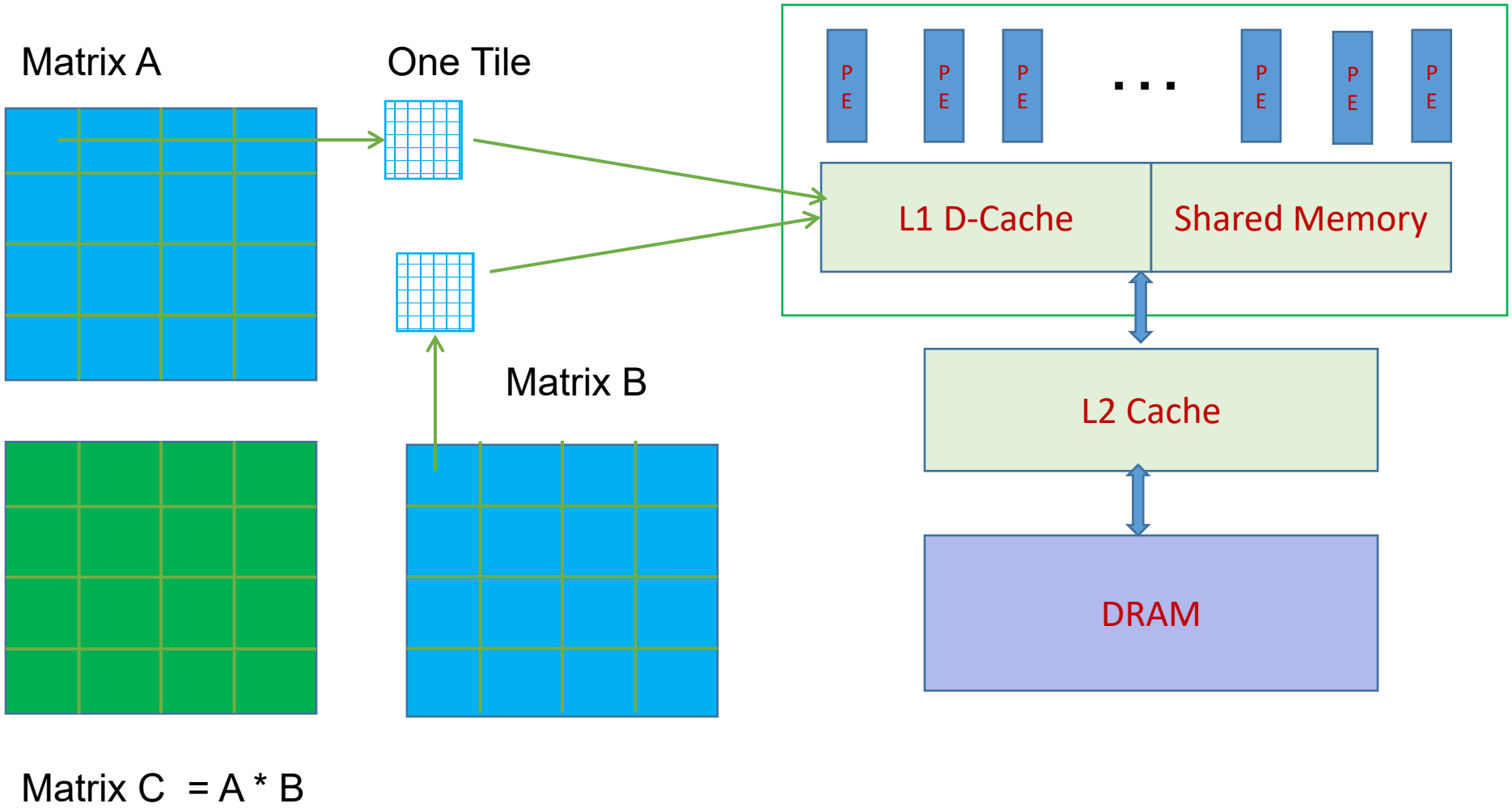
Code Optimization

Two Most Important Code Optimizations

- Tiling or blocking
  - Goal:
    - Make the working set of a TB (or multiple TBs) to fit in shared memory or D-cache;
    - Convert un-coalesced accesses into coalesced ones.

- Thread Coarsening/Fusion/Merge
  - Goal: data reuse and removal of redundant instructions

Matrix-Multiplication



Matrix Multiplication: Tiling

- CPU code

Untiled version

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++){
        C[i][j] = 0;
        for(k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
```



Tiled version

```
for (i = 0; i < N; i+=Nb)
    for (j = 0; j < N; j+=Nb)
        for(ii = i; ii < i + Nb;ii++)
            for(jj = j; jj < j+Nb; jj++){
                C[ii][[jj] = 0;
                for(k = 0; k < N; k+= Nb)
                    for(kk = k; kk < k+Nb; kk++)
                        C[ii][jj] += A[ij][kk] * B[kk][jj];
    }
```

Matrix Multiplication: Tiling

- GPU kernel code

```
Tiled version
for (i = 0; i < N; i+=Nb)
   for (j = 0; j < N; j+=Nb)
      for(ii = i; ii < i + Nb;ii++)
          for(jj = j; jj < j+Nb; jj++){
             C[ii][[jj] = 0;
             for(k = 0; k < N; k+= Nb)
               for(kk = k; kk < k+Nb; kk++)
                  C[ii][jj] += A[ij][kk] * B[kk][jj];
   }
```

Two-dimension Grids

Two-dimension TB



```
for (int a = aBegin, b = bBegin; a <= aEnd; a += aStep, b += bStep)
{
     __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
     __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];
      AS(ty, tx) = A[a + WA * ty + tx];  //tx = threadIdx.x
      BS(ty, tx) = B[b + WB * ty + tx]; //tx = threadIdx.y
       __syncthreads();
  #pragma unroll
       for (int k = 0; k < BLOCK_SIZE; ++k)
            Csub += AS(ty, k) * BS(k, tx);
       __syncthreads();
  }
```

Performance Analysis

- Roofline model
  - Computation or bandwidth bound?

- Memory performance analysis
  - (global) memory coalescing
  - Shared memory bank conflicts

AI (Arithmetic Intensity) improvement from Tiling

- Matrix multiplication
- Realistic on-chip caches: small caches wrt matrix sizes

$AI_{beforetiling}$ =

$AI_{aftertiling}$ =

Thread Coarsening/Fusion/Merge

- Combine/Fuse/Merge multiple threads' workload into one
  - E.g., in matrix multiplication, one thread computes 1 output element. With thread coarsening, one thread computes 2 or more elements.

Original version

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++){
        C[i][j] = 0;
        for(k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
    }
```

Fused version (two elements in the same column)

```
for (i = 0; i < N; i+=2)
    for (j = 0; j < N; j++){
        C[i][j] = 0;
        C[i+1][j] = 0;
        for(k = 0; k < N; k++)
            C[i][j] += A[i][k] * B[k][j];
            C[i+1][j] += A[i+1][k] * B[k][j];
    }
```

Matrix Multiplication After Thread Coarsening

- GPU Code

```
__global__ void MatMul(float*A, float* B, float * C, int N) {
      int j = blockIdx.x * blockDim.x + threadIdx.x; //idx
      int i = blockIdx.y * blockDim.y + threadIdx.y; //idy
      i = i * 2;
      sum1 = 0; sum2= 0;
     for(k = 0; k < N, k++){
           sum1 += A[i*N+k] * B[k*N + j];
           sum2 += A[(i+1)*N + k] * B[k* N + j];
     }
    C[i*N + j] = sum1; C[(i+1)*N + j] = sum2;
 }
```

Host code: dim3 numBlocks(N / threadsPerBlock.x, N / threadsPerBlock.y / **2**);
MatMul<<<numBlocks, threadsPerBlock>>>(A, B, C, N);

Tiling and Coarsening

- Orthogonal optimizations
- Often can be used together
  - E.g., Matrix multiplication in CUBLAS.